

Risoluzione Circuiti Elettrici

Implementazione C++ del Metodo delle Correnti di Maglia

Gruppo di Progetto

Campagnoli Gianluca	(Matricola 326224)
Di Carlo Michele	(Matricola 326240)
Anzaldi Noemi	(Matricola 325034)

Anno Accademico 2025/2026

Indice degli Argomenti

- 1 Architettura dei Componenti
- 2 Gestione dell'Input
- 3 Astrazione Topologica
- 4 Analisi delle Criticità e Sicurezza
- 5 L'Algoritmo di De Pina e Calcolo Numerico

Difficoltà Incontrata:

Nel Metodo delle Maglie, Generatori e Resistori hanno ruoli matematici profondamente diversi. I resistori compongono la matrice di incidenza B e la matrice delle resistenze R , mentre i generatori finiscono solo nel vettore dei termini noti v .

Approccio Iniziale Scartato:

Creare un'unica classe `Edge` con un flag booleano `is_generator`. Questo avrebbe costretto a inserire continui `if (edge.is_generator)` in ogni ciclo di calcolo matriciale, diminuendo le performance.

Scelta Implementativa: Template e Vettori Separati

- Abbiamo creato classi distinte `generator` e `resistor`.
- Abbiamo definito l'arco come template: `undirected_edge<T>`.
- Nel grafo, conserviamo due strutture separate:
`vector<undirected_edge<resistor>>` e
`vector<undirected_edge<generator>>`.

I Vantaggi della Scelta:

- **Efficienza di Calcolo:** Per costruire la matrice delle resistenze R , il codice itera *esclusivamente* sui resistori, senza dover controllare ed escludere i generatori a ogni ciclo.
- **Linearità del Codice:** Totale assenza di costrutti condizionali (`if`) all'interno dei loop algebrici pesanti

Difficoltà Incontrata:

Nella traccia viene specificato: *si stabilisce che il verso di percorrenza dell'arco è sempre dal nodo di indice minore a quello di indice maggiore.*

Scelta Implementativa

- Noi questa regola l'abbiamo applicata solo alle resistenze utilizzando una funzione `node_sort()` che switcha i nodi se il nodo `_from` è maggiore del nodo `_to`.
- Per i generatori, invece, non eseguiamo l'ordinamento perché è fondamentale preservare l'ordine originale scritto nel file di input per non alterarne la polarità.

Astrazione: Unificazione per la Topologia

Difficoltà Incontrata:

Se i componenti sono in vettori separati, come facciamo ad applicare l'algoritmo di De Pina sull'intero grafo senza scrivere algoritmi duplicati?

Scelta Implementativa:

L'introduzione della classe `UniversalEdge` che unisce le diverse classi di componenti.

Mentre i vettori tipizzati servono per l'**Algebra Lineare**, il vettore unificato `UniversalEdge` serve per la **struttura delle connessioni del grafo (Topologia)**. Gli algoritmi più avanzati (come De Pina) funzionano su qualsiasi tipo di `UniversalEdge`. In questo modo il codice si scrive una volta sola e si riutilizza ovunque.

Difficoltà Incontrata (Analisi del Limite):

Possibilità di avere una Netlist in cui i nodi non sono sequenziali (1, 2, 3, ecc...) oppure negativi (-1, -2, -10, ecc...). In questi casi l'accesso tramite un vettore (reached) che memorizza i nodi visitati con l'operatore [] darebbe *Segmentation Fault*.

Scelta Implementativa:

Abbiamo optato per `std::unordered_map<int, bool> reached`. Alloca memoria solo per i nodi reali esistenti; questo protegge il parser da dati imprevisti ed evita blocchi o crash della memoria.

Difficoltà Incontrata:

Circuiti paralleli rendono degenerare il grafo per l'algoritmo matematico. Poiché il grafo è *non orientato*, l'inserimento dell'arco (1, 2) e (2, 1) rappresenta fisicamente la stessa connessione.

Scelta Implementativa:

Implementazione di una verifica per evitare duplicati durante il *parsing*:

- La funzione `has_same_nodes` verifica incrociando soltanto i nodi `from` e `to`, senza preoccuparsi della componente.
- Respinge componenti paralleli in tempo reale, interrompendo la generazione del grafo **prima** che provochi instabilità nei solver iterativi.

Difficoltà Incontrata:

L'algoritmo di De Pina per la base dei cicli minimi richiede il calcolo di cammini su un *lifting* del grafo a due piani (G' con nodi positivi e negativi). Duplicare fisicamente la struttura del grafo avrebbe raddoppiato lo spazio in memoria.

Scelta Implementativa:

Abbiamo unito l'esplorazione alla macchina a stati:

- Lo stato è definito come una coppia: `std::pair<int, int>` (nodo, piano).
- La visita BFS "salta" di piano matematicamente sfruttando il vettore di incidenza S_j come flag di inversione.

Difficoltà Incontrata (Stabilità Numerica):

Il sistema $B^T R B i = v$ è risolto col Gradiente Coniugato. A causa dell'aritmetica floating-point, il residuo $\mathbf{r} = \mathbf{b} - A\mathbf{x}$ non sarà mai perfettamente nullo. Serve un criterio di arresto robusto rispetto alle grandezze in gioco.

Scelta Implementativa: Tolleranza Ibrida

- **Tolleranza Relativa (1.0e-10):** Il ciclo termina quando $\|\mathbf{r}_k\| \leq \text{res_tol} \cdot \|\mathbf{r}_0\|$. L'errore viene scalato in base alle magnitudini iniziali.
- **Tolleranza Assoluta (1.0e-16):** Controllo preventivo che rileva sistemi degeneri o vettori innesco quasi nulli, prevenendo divisioni per zero.

Sintesi delle Scelte Implementative

- **Organizzazione a moduli:** Serve a non scrivere due volte le stesse cose e a tenere divisi il disegno del circuito (topologia) e la matematica (algebra lineare). Così i calcoli funzionano a prescindere da come è fatto il circuito.
- **Robustezza:** Sostituzione di array statici con mappe dinamiche per prevenire buffer overflow su input critici.
- **Sicurezza Numerica:** Controlli preventivi sui circuiti paralleli e gestione delle tolleranze per blindare la convergenza.

Utilizzo dell'Intelligenza Artificiale

- **Testing e Validazione:** Generazione di Netlist *ad hoc* per lo stress-test del codice, verificando i calcoli tramite il confronto incrociato tra l'output dell'AI e i simulatori in rete.
- **Analisi della Stabilità Numerica:** Supporto nella scelta della *tolleranza relativa* nel solutore Gradiente Coniugato ($1.0e-10$), che è quella più utilizzata in letteratura

Grazie per l'attenzione!

Campagnoli Gianluca - matricola 326224
Di Carlo Michele - matricola 326240
Anzaldi Noemi - matricola 325034